On the Reliability and Explainability of Language Models for Program Generation

YUE LIU, CHAKKRIT TANTITHAMTHAVORN, and YONGHUI LIU, Monash University, Clayton, Australia
LI LI, Beihang University, Beijing, China

Recent studies have adopted pre-trained language models, such as CodeT5 and CodeGPT, for automated program generation tasks like code generation, repair, and translation. Numerous language model based approaches have been proposed and evaluated on various benchmark datasets, demonstrating promising performance. However, there is still uncertainty about the reliability of these models, particularly their realistic ability to consistently transform code sequences. This raises a question: are these techniques sufficiently trustworthy for automated program generation? Consequently, further research is needed to understand model logic and assess reliability and explainability. To bridge these research gaps, we conduct a thorough empirical study of eight popular language models on five representative datasets to determine the capabilities and limitations of automated program generation approaches. We further employ advanced explainable AI approaches to highlight the tokens that significantly contribute to the code transformation. We discover that state-of-the-art approaches suffer from inappropriate performance evaluation stemming from severe data duplication, causing overoptimistic results. Our explainability analysis reveals that, in various experimental scenarios, language models can recognize code grammar and structural information, but they exhibit limited robustness to changes in input sequences. Overall, more rigorous evaluation approaches and benchmarks are critical to enhance the reliability and explainability of automated program generation moving forward. Our findings provide important guidelines for this goal.

CCS Concepts: • **Software and its engineering → Software maintenance tools**; • **General and reference → Reliability**; • **Computing methodologies → Natural language processing;**

Additional Key Words and Phrases: Automated program generation, empirical analysis, explainable AI

ACM Reference Format:
Yue Liu, Chakkrit Tantithamthavorn, Yonghui Liu, and Li Li. 2024. On the Reliability and Explainability of Language Models for Program Generation. *ACM Trans. Softw. Eng. Methodol.* 33, 5, Article 126 (June 2024), 26 pages. https://doi.org/10.1145/3641540

1 INTRODUCTION

Program generation (e.g., code repair and code translation) involves producing new source code sequences to facilitate software development and maintenance. These activities require software

C. Tantithamthavorn was supported by the Australian Research Council's Discovery Early Career Researcher Award (DECRA) funding scheme (DE200100941).
Authors' addresses: Y. Liu, C. Tantithamthavorn (Corresponding author), and Y. Liu, Monash University, Wellington Road, Clayton, Victoria, Australia, 3800; e-mails: yue.liu1@monash.edu, chakkrit@monash.edu, Yonghui.Liu@monash.edu; L. Li, Beihang University, 37 Xueyuan Road, Beijing, China, 100191; e-mail: lilicoding@ieee.org.

ACM 1049-331X/2024/06-ART126
https://doi.org/10.1145/3641540

engineers to manually analyze, understand, and even execute programs to ensure the quality of newly developed code. Thus, automated learning based program generation has been proposed to automatically generate new code based on context. Recently, language models have shown promise in semantic modeling and natural language understanding [7, 75]. Due to similarities between text and source code [12], language models (e.g., LSTM and Transformer) have gained immense research interest for automated code generation and understanding [37, 72]. Specifically, pre-trained language models for code (e.g., CodeT5 [81] and CodeBERT [19]), typically pre-trained on massive unlabeled code corpora using self-supervised objectives, can learn generic source code representations. These can then be transferred to diverse downstream tasks, decreasing pre-training costs and improving performance on code understanding and generation [78]. Consequently, pre-trained language models have become the state of the art for many code intelligence tasks, such as code review [28, 38], code summarization [76], code completion [39, 76], and program repair [87].

Despite the effectiveness and advanced capabilities of language models for code demonstrated in prior work, these powerful program generation models are not yet reliable enough for practical application [63, 84]. For example, prior studies [62, 83, 84] have shown that language models are extremely vulnerable to adversarial attacks, where attackers can induce incorrect outputs through minor perturbations. Additional analysis and demonstration of model robustness are clearly needed. Furthermore, impressive results in lab-only settings may not translate to real-world efficacy [63]. Next, we discuss three significant issues and implications that must be addressed before widespread adoption of program generation models.

Insufficient Interpretability Analysis. One severe drawback of prior research is a lack of explainability analysis on language models for code. In contrast to linear models, language models have complex architectures (e.g., standard CodeT5 with 12 layers, 220M parameters), making it challenging to explain specific predictions [14, 67–69]. In other words, the recommendations made by language models are opaque to practitioners and researchers, who are unlikely to accept code recommendations without evidence, especially for non-trivial cases [33, 48, 60]. For example, although prior studies have proven that language models can automatically update code to fix bugs, which input characteristics contribute to the prediction is largely overlooked [84, 87]. This gap in interpretability analysis significantly hinders the real-world adoption of program generation models.

Experimental Bias. While prior research has demonstrated the effectiveness of language models on code-based tasks, their performance evaluations can be susceptible to experimental bias [84]. A common issue is dataset bias [9, 65, 86], where most code snippets are collected from open source projects in a compromised manner that potentially introduces noise. For example, Zhang et al. [86] trained Transformer-based methods on a noisy dataset containing both bot-generated and trivial code commit messages, achieving a 42.4% BLEU-4 score. However, removing the noisy data resulted in a sharp performance drop to 26.2% BLEU-4. Therefore, the actual capabilities of language models trained on such biased datasets are difficult to accurately evaluate. They likely face severe performance degradation when applied in practice versus lab settings.

Poor Practicability. Substantial research indicates promise for using language models in automated program generation; however, most evaluation results demonstrate that these techniques are not yet practical for real-world application. For example, Tufano et al. [72] proposed a Transformer-based automated code change approach, achieving just 21.16% accuracy. With an accuracy of 21% (i.e., successfully predicted code transformations), there are too many false negatives, which means that practitioners can never know whether the code update fits or not. Further experiments showed that increasing the beam search size from 1 to 10 brings a significant performance improvement (i.e., 21.16% to 36.02%). However, picking the best sequence among 10 candidates remains challenging for practitioners.

To the best of our knowledge, no systematic research study has investigated the reliability and explainability of language models for program generation. In response to the observations and concerns raised previously, we conduct the first comprehensive empirical evaluation of popular pre-trained language models on automated program generation tasks. Specifically, we adopt eight mainstream pre-trained models: T5 [61], CodeT5 [81], CoTexT [58], CodeTrans [17], CodeGPT [44], CodeBERT [19], CodeT5+ [79], and CodeReviewer [38]. We evaluated performance on four popular program generation tasks—code repair, code review, code translation, and text-to-code generation—using five state-of-the-art benchmark datasets (i.e., *Tufano et al. [72]*, *Bugs2Fix [73]*, *CodeReview [38]*, *CodeTrans-Dataset [44]*, and *CONCODE [31]*). Our results confirm the superior accuracy of studied models reported in prior work, with some achieving nearly 70% on certain datasets. However, we discovered that performance is skewed by inappropriate experimental design, yielding unreliable and unrealistic evaluations. Specifically, duplication in benchmark datasets skews results. Some have explicit duplicates between training and testing sets, directly inflating performance. In addition, even where testing sets lack identical training examples, they may contain duplicated testing cases. This overlapping test data further distorts evaluation. For code-to-code tasks, a concerning observation is that a substantial percentage of generated code exactly matches inputs, instead of generating refined or updated code sequences. Moreover, explanation results suggest that program generation models can recognize code grammar and structural information. However, they present poor robustness even to minor changes in input sequences. In contrast to previous research, our findings indicate that automated program generation remains imperfect, with ample room for improvement through future work.

Contribution. The main contributions of this article are summarized as follows:

- To the best of our knowledge, we conduct the first comprehensive benchmark study of pre-trained language models for program generation, evaluating reliability and explainability.
- Our analysis reveals significant experimental biases in prior work, including dataset duplication and overlapping inputs that inflate performance claims. Explanation analysis demonstrates that models overlook critical tokens and lack robustness, highlighting key challenges for practical deployment.
- Results provide insights to guide future research toward more rigorous and reliable language models for neural program generation.

Open Science. To support the open science initiative, we publish the studied dataset and a replication package, which are publicly available on GitHub.[1]

Article Organization. The rest of the article is structured as follows. Section 2 presents the background. Section 3 details our experimental design. Section 4 provides our experimental results and analysis. Section 5 discusses the study's implications and potential threats to its validity. Related work is briefly introduced in Section 6, followed by a conclusion in Section 7.

2 BACKGROUND

2.1 Automated Program Generation

Automated program generation exploits language models to analyze and emulate the distribution patterns of both source code and natural text, enabling the generation of new code snippets. As a result of the similarity of the data representation, program generation models inherit many techniques from natural language processing. As a result, a variety of techniques from natural language generation, especially those involving Transformer-based models, have found increasing application in automated code-based tasks [18, 29]. In addition, language models like T5 [61] have

[1] https://github.com/yueyueL/ProgramGen-LMs-Reliability

demonstrated the ability to effectively learn code representation from unlabeled data to conduct a wide range of downstream tasks given supervised discriminative fine-tuning on specific tasks (e.g., code completion [39, 66, 76], code search [27, 65], code summarization [76], code review [38, 59, 70–72], API recommendation [10], and vulnerability analysis [20–25]).

In this work, our focus is exclusively on automated program generation tasks that involve the transformation of a source code snippet or natural language into a new or modified program snippet, since their strengths and limitations have not been thoroughly studied. We describe program generation models for the task of mapping a source sequence $X = [x_1, \ldots, x_M]$ (where M represents the length of the source sequence) to a target sequence $Y = [y_1, \ldots, y_N]$ (where N signifies the target sequence length). Specifically, the raw source sequences will be first preprocessed and mapped into a sequence of embeddings $(x_1, \ldots, x_M)$. Next, the Transformer-based models jointly train the encoder and decoder for comprehensive source code modeling. In particular, program generation models are usually composed of encoder layers and decoder layers, where the encoder takes a sequence of code tokens as input to map an initial method $X = [x_1, \ldots, x_M]$ into a fixed-length intermediate hidden state $H = [h_1, \ldots, h_M]$. Then, the decoder takes the hidden state vector H as an input to generate the output sequence of tokens $Y = [y_1, \ldots, y_N]$. We note that M (i.e., the length of the input sequence) and N (i.e., the length of the output sequence) can be different. To optimize the mapping, the parameters of the model are updated using the training dataset with the following equation to maximize the conditional probability:

$$p(Y \mid X) = p(y_1, \ldots, y_m \mid x_1, \ldots, x_n) = \prod_{i=1}^{m} p(y_i \mid H, y_1, \ldots, y_{i-1}).$$

Many pre-trained language models (e.g., T5 [61], CodeT5 [81], and CoTexT [58]) have been proposed and studied on various benchmarks and datasets. These pre-trained models decrease the cost of pre-training a large-scale language model from scratch and improve the performance of various source code understanding and generation tasks [78]. In other words, pre-trained code language models can be fine-tuned on different datasets for specific tasks, making them a state-of-the-art architecture in automated program generation.

2.2 Model Explanation

Interpreting automated program generation models is critical for practitioners and researchers to comprehend and refine the decision-making processes inherent to these models. The application of explainable AI approaches serves to illustrate the learned behaviors of these models.

Explainable Program Generation. Consider an automated program generation model f that transforms a source sequence $X = (x_1, \ldots, x_M)$ into a predicted code sequence $Y = (y_1, \ldots, y_N)$, where M and N represent the lengths of the input and output sequences, respectively. The goal of an explainable AI approach is to ascertain the relevance r_i of each token within the context of both the input and the sequence of tokens generated thus far. This relevance can be formalized as $r_i = (a^1, \ldots, a^M, b^1, \ldots, b^{i-1})$, where a^k represents the importance of the k-th input token and b^j signifies the influence of the j-th token in the existing output sequence upon the generation of the next token y_i. For each token y_i in the predicted sequence Y, its feature importance is reflected not just by the input tokens but also by the contribution of all preceding tokens in the output sequence.

Figure 1 presents an example of explainable AI applied to program generation. It displays the model's reasoning when generating the "setTitle" token, taking into account both the original input and the previously generated tokens. As depicted, tokens such as "setTitle" and "title" from the source sequence, and "void" from the existing generated tokens, are deemed highly important in

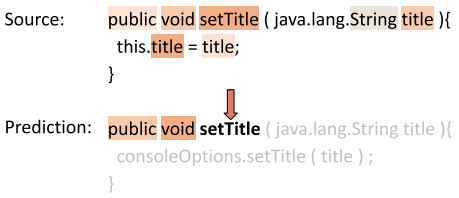

Fig. 1. An explanation example for code review tasks, illustrating the generation of the "setTitle" token from a given source code sequence.

this context. This visualization reinforces the sequence-to-sequence nature of program generation, where each token is generated in consideration of both the preceding and subsequent context. In this section, we employ interpretable AI approaches that enable us to quantify the significance of each token conversion from the source code to the generated program sequence. By doing so, we aim to unpack the decision-making processes of program generation models, thereby offering insights into their predictive behavior.

2.2.1 Attention-Based Analysis. Most state-of-the-art pre-trained language models are based on the Transformer architecture [75], with the self-attention mechanism. The attention mechanism, which provides a distribution of scores over the input tokens, has often been presented as showing the relative importance of the inputs. Specifically, the higher the attention weights, the more attention that is paid by the model. Therefore, there have been many prior studies that employ the attention weights of pre-trained programming language models to explain model predictions [76, 80, 88]. Prior studies calculate the feature importance of each token by averaging the attention weights of all layers and heads. However, variations in attention across different heads and layers, as noted by Wan et al. [76], suggest that each attention head focuses differently on various aspects of the source code, raising questions about the efficacy and accuracy of attention weights as explanatory tools. Given the complexity and layered nature of pre-trained program generation models, and the ambiguity in determining the saliency of different attention weights for model predictions, we employ a model-agnostic interpretable approach in this study.

2.2.2 SHAP. SHAP (i.e., SHapley Additive exPlanations) [45] is a popular black-box model-agnostic interpretable approach. SHAP treats the candidate model f as a black box. SHAP utilizes Shapley values, a game theory based approach, to approximate the relationship between the input and the output prediction. Contrary to attention analysis, SHAP provides one feature importance vector to explain the whole relationship between source and target sequences. Consequently, we use the SHAP approach to analyze and understand language models in this study. SHAP provides multiple explainable methods to generate SHAP values as an explanation. In this study, we apply GradientExplainer to language model based program generation models. GradientExplainer is an extension of the integrated gradients approach and approximates SHAP values by computing the expectations of gradients by randomly sampling from the distribution of baseline/references.

3 EXPERIMENTAL DESIGN

In this section, we describe the setup and methodologies of our empirical study, focusing on the reliability and explainability of language model based program generation.

3.1 Language Models

Recently, pre-trained language models for automated code understanding and generation tasks have been studied extensively in academia and industry. These existing approaches can be

categorized into three types: encoder-based models such as CodeBERT [19], decoder-based models such as CodeGPT [44], and encoder-decoder-based models like CodeT5 [81]. Although prior works [8, 38, 84] have proven that encoder-based models and decoder-based models are not good at generation tasks, we include them in our study for completeness. While there can be many potential state-of-the-art models to be studied, we chose eight representative pre-trained language models (i.e., T5 [61], CoTexT [58], CodeTrans [17], CodeBERT [19], CodeGPT [44], CodeT5 [81], CodeReviewer [38], and CodeT5+ [79]) that are publicly available for evaluation.

***T5* [61].** Raffel et al. [61] proposed the T5 (Text-To-Text Transfer Transformer) architecture, pre-trained on a large natural language corpus. T5 has demonstrated state-of-the-art performance when fine-tuned for many NLP tasks. As T5 is only pre-trained on natural language data, we utilize it as a baseline to compare against pre-trained models for programming language.

***CoTexT* [58].** CoTexT utilizes the same architecture as T5 and is pre-trained on the CodeSearchNet Corpus [30] and Google BigQuery [1]. CoTexT has achieved state-of-the-art results on code generation benchmarks [58].

***CodeTrans* [17].** CodeTrans is a large pre-trained Transformer-based encoder-decoder inspired by T5. Since the official online repository provides many versions, we use the version with the most downloads. The CodeTrans model we used in this study is trained on 9,714 Java open source projects from GitHub.

***CodeBERT* [19].** CodeBERT is an encoder-based model pre-trained on natural language and programming language corpora using RoBERTa architecture. CodeBERT is pre-trained on CodeSearchNet Corpus [30]. It has demonstrated strong performance on code search and summarization [84].

***CodeGPT* [81].** CodeGPT is a decoder-only transformer model pre-trained on multiple programming languages. CodeGPT is designed for code generation tasks like method name prediction, code completion, and code translation [44].

***CodeT5* [81].** CodeT5 is a state-of-the-art unified pre-trained encoder-decoder programming language model. Inspired by T5, Wang et al. [81] pre-trained the T5 architecture on eight programming languages together with their comments collected from open source repositories. CodeT5 has demonstrated promising performance on code-related generation tasks [23, 28, 36].

***CodeReviewer* [38].** CodeReviewer is built on CodeT5 and is pre-trained on a large dataset of code updates and corresponding comments in code review scenarios. CodeReviewer has proven prominent performance on code refinement by recent works [38].

***CodeT5+* [79].** CodeT5+ is an enhanced version of CodeT5, introducing architectural enhancements and advanced pre-training techniques like span denoising and contrastive learning. These innovations enable CodeT5+ models to achieve state-of-the-art performance across diverse code intelligence tasks, even zero-shot text-to-code generation [79].

3.2 Downstream Tasks and Corresponding Datasets

Program generation tasks involve producing or generating sequences of tokens in a programming language. To better understand why language models perform program generation effectively, we extensively evaluate four different scenarios, in five datasets, for automated program generation as shown in Table 1. The details of each scenario are described next.

Code Review. The objective of code review is to automatically implement code changes by developers during pull requests, including bug fixing, refactoring, and optimization. Code review is a critical part of software development and serves as a common program generation task, extensively examined in prior research [38, 70, 72]. In our empirical study, we assess two different datasets related to code review tasks:

Table 1. Summary of Datasets Used for Different Tasks

Task	Subsets	Category	Language	Dataset Size
Android_S, *Android_M*, *Google_S*, *Google_M*, *Ovirt_S*, *Ovirt_M*	Code Review	Code-Code	Java	21,774
CodeReview	Code Review	Code+Comment-Code	Java, Python, Go, C++, C, C#, JavaScript, Php, Ruby	1.3M
B2F_S, *B2F_M*	Code Repair	Code-Code	Java	123,805
Java2C#, *C#2Java*	Code Translation	Code-Code	Java, C#	11,500
CONCODE	Code Generation	Text-Code	Java	104,000

— *Tufano et al.*: Tufano et al. [72] collected code review data from three large Gerrit [2] code review repositories (i.e., *Android*, *Google*, and *Ovirt*). This collection forms a code-to-code corpus, where the source code prior to the pull request is transformed into the target code, reflecting the changes post-review. The dataset is segregated according to method length into 10,783 *Small* instances (with fewer than 50 tokens) and 10,991 *Medium* instances (with 50 to 100 tokens). Consequently, six subsets are obtained: *Android_S*, *Android_M*, *Google_S*, *Google_M*, *Ovirt_S*, and *Ovirt_M*.

— *CodeReview*: Li et al. [38] collected pull request data from the top popular open source projects on GitHub. This expansive dataset mines the projects in nine programming language data, resulting in a collection of approximately 1.3 million pairs of pre- and post-review code. The dataset forms a "code+text to code" corpus, where the input is the initial code along with associated commits, and the output is the revised code following the pull request.

Code Repair. Code repair aims to fix bugs in the code automatically. This is one of the most common program generation tasks and has been widely investigated by prior work [44, 73]:

— *Bugs2Fix*: Tufano et al. [73] systematically extracted bug-fixing commit data from a large number of GitHub repositories, obtaining method-level pairs of buggy and fixed Java code snippets. This dataset presents a code-to-code transformation task, where the source is the buggy code and the target is the fixed code. The dataset is split into two subsets based on the code length, *B2F_S* for small fixes (≤50 tokens) and *B2F_M* for medium fixes (>50 and ≤100 tokens). Respectively, these subsets encompass 58,350 small and 65,455 medium bug-fix instances.

***Code Translation* [50].** This task aims to translate code from one programming language to another while preserving its functionality:

— *CodeTrans-Dataset*: *CodeTrans-Dataset* [44], part of the CodeXGLUE benchmark [44], provides paired examples for code translation between Java and C#, culminating in a total of 11,800 functionally equivalent method pairs. This dataset enables the evaluation of translation from Java to C# and vice versa, resulting in two distinct datasets: *Java2C#* and *C#2Java*.

Code Generation. Code generation refers to the creation of executable code based on natural language descriptions:

— *CONCODE*: The *CONCODE* [31] dataset is widely used in code generation research and collects examples from approximately 33,000 Java projects on GitHub. It encompasses 100,000 examples for training along with 4,000 examples for validation and testing. Each example is composed of a triple: a natural language description, code environments, and code snippets. This dataset exemplifies the "text-to-code" challenge, emphasizing the need for algorithms capable of understanding textual descriptions and transforming them into syntactically and semantically correct code.

In summary, our empirical study spans multiple task scenarios including code review, code repair, code translation, and code generation. We have obtained 12 distinct subsets for this purpose: *Android_S*, *Android_M*, *Google_S*, *Google_M*, *Ovirt_S*, *Ovirt_M*, *CodeReview*, *B2F_S*, *B2F_M*, *Java2C#*, *C#2Java*, and *CONCODE*. These datasets facilitate the exploration of three input-output paradigms: code-to-code, code+text-to-code, and text-to-code, each pivotal to the domain of program generation.

3.3 Evaluation Metrics

To evaluate model performance, we primarily use accuracy (i.e., exact match rate or perfect prediction rate), which is the commonly used metric for program generation tasks [38, 70, 72, 73]. Specifically, accuracy is calculated by dividing the number of perfectly predicted code snippets by the total snippets in the test set. Additionally, we incorporate the BLEU-4 score from the NLP domain [56] to evaluate the similarity between the generated and target code. BLEU helps evaluate partial matches and overall fluency, complementing the exact match rate. To further analyze potential data duplication within datasets, we introduce a modified BLEU-4 metric to quantify dataset-level similarity. This enables identifying and measuring repetitive patterns that could skew model training and evaluation. For explanation results, we analyze and compare the average feature importance vectors $\bar{r}$. This provides insight into how models utilize input features for prediction.

3.4 Implementation

We build language models on top of two Python libraries: PyTorch [57] and Transformers [82]. The studied models from Section 3.1 are obtained via the Transformers API. To enable fair comparison, we use the 12-layer base version for all models, consistent with common practice in relevant literature [38, 84]. In this study, encoder-based models (CodeBERT) are appended with a transformer decoder to generate code regressively. Decoder (CodeGPT) and encoder-decoder-based models (CodeT5) directly generate code aggressively, as adopted by prior studies [44, 84]. For each dataset, we use identical training, validation, and test splits as well as fine-tuning approaches described in CodeXGLUE [44] and original papers. The *Bugs2Fix*, *CodeTrans-Dataset*, and *CONCODE* datasets were collected by CodeXGLUE, a benchmark for program understanding and generation. The *Tufano et al.* and *CodeReview* datasets are in original format. Regarding our explorations into explainable AI, we adopt the implementations provided by the Captum [34] and Ecco [4] packages, both of which are designed for Python. Experiments are run on a machine with an AMD Ryzen 9 5950X 16-core @ 3.4-GHz CPU, 64 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU with 24 GB of memory.

3.5 Research Questions

We structure our study around three key research questions to comprehensively evaluate language models for program generation:

RQ1: How Do Language Models Perform on Program Generation Tasks? Numerous pretrained language models have been proposed, with some demonstrating promising capabilities on certain tasks. However, a systematic and extensive exploration of performance across diverse datasets and models is lacking. We undertake this broad investigation, evaluating the latest state-of-the-art models on tasks spanning code repair, review, translation, and generation. This research question aims to undertake a comprehensive evaluation of language models, examining their ability to not only replicate previous success but also generalize across distinct program generation tasks and datasets.

RQ2: How Reliable Are Automated Program Generation Approaches? This research question critically analyzes the evaluation approaches used to assess automated program generation models,

Table 2. Performance of Language Models on the Studied Program Generation Datasets (Accuracy)

	Android_S	Android_M	Google_S	Google_M	Ovirt_S	Ovirt_M	CodeReview	B2F_S	B2F_M	Java2C#	C#2Java	CONCODE
T5	6.42%	2.75%	4.93%	1.11%	16.85%	9.65%	18.00%	15.15%	5.58%	54.40%	62.40%	20.05%
CoTexT	7.50%	3.98%	5.20%	1.80%	16.64%	10.99%	20.18%	16.61%	6.40%	57.00%	65.70%	20.70%
CodeTrans	7.02%	3.30%	4.23%	0.97%	11.96%	7.12%	8.88%	8.14%	2.60%	49.10%	52.10%	21.65%
CodeBERT	8.50%	5.01%	3.17%	0.83%	14.64%	10.13%	21.03%	12.10%	4.19%	50.20%	54.70%	18.60%
CodeGPT	11.37%	8.14%	8.99%	4.50%	19.92%	12.05%	16.70%	13.61%	4.64%	59.90%	64.50%	17.65%
CodeT5	13.80%	9.21%	10.40%	4.82%	22.06%	12.64%	28.31%	17.60%	8.34%	64.10%	69.90%	22.35%
CodeReviewer	14.68%	10.40%	11.81%	6.85%	25.49%	18.18%	30.43%	17.94%	8.77%	63.10%	70.40%	22.65%
CodeT5+	15.20%	11.36%	14.27%	7.27%	26.06%	20.03%	30.12%	18.44%	7.84%	63.90%	70.60%	21.85%

to identify potential experimental flaws or biases that could undermine performance evaluation. Specifically, we investigate the representativeness and diversity of training and testing datasets. Performance heavily relies on dataset quality (i.e., "garbage in, garbage out") [63]. Widespread duplication or lack of diversity could skew results. In this research question, we aim to uncover potential limitations that lead to overestimating or underestimating realistic capabilities. Addressing these concerns is vital for establishing robust and reliable benchmarking practices that will contribute to accurate characterization and continued improvement of automated program generation approaches.

RQ3: Can We Explain Why Automated Program Generation Approaches Can (or Fail to) Generate Code Sequences Reliably? Only analyzing the generated sequences still does not determine why language models perform effectively or ineffectively. The main reason is that what these pre-trained language models are based on to predict new code sequences is largely unknown. Therefore, we employ explainable AI approaches to understand what tokens contribute to the generated code sequences. We expect our exploratory experiments on explainable automated program generation can put forward practical insights for future research. We employ a state-of-the-art model-agnostic explainable approach for interpreting language models. We then utilize the explanation results to understand why program generation models output new code sequences effectively or ineffectively.

4 RESULT

To answer the aforementioned research questions, we perform an extensive analysis of the generated code sequences and their explanation results, respectively. Next, we present the results with respect to our three research questions.

4.1 RQ1: How Do Language Models Perform on Program Generation Tasks?

Approach. To better understand how well the pre-trained language models perform in automated program generation, we extensively study and compare their performance on the studied datasets. We follow the same training/validation/testing data splits and fine-tuning procedure for each dataset introduced in Section 3.2. We employed a beam search setting of 1 for our evaluations. Then, we calculated the accuracy (i.e., the exact match rate) for each model. Table 2 presents a detailed summary of the accuracy achieved by eight different language models across these datasets.

Result. We observe from Table 2 several key findings on language model performance on program generation tasks. First, we find that language models exclusively pre-trained on natural language corpora, such as T5, demonstrate limited effectiveness on programming tasks compared to models pre-trained on code data. For example, T5 achieves just 6.42% and 2.75% accuracy on *Android_S* and *Android_M*, respectively, highlighting the importance of domain-specific pre-training in improving performance on automated program generation. Additionally, we observe that model architecture plays a significant role in performance. Decoder-only (CodeGPT) and encoder-only (CodeBERT) models exhibit inferior results across multiple datasets compared to encoder-decoder architectures like CodeT5. Furthermore, we observe that models such as CodeT5, CodeReviewer, and CodeT5+

consistently outperform others across various benchmarks. For example, on the code+comment-to-code *CodeReview* dataset, CodeReviewer obtains state-of-the-art performance with 30.43% accuracy. Similarly, it leads text-to-code performance on the *CONCODE* dataset with 22.65% accuracy. In other code-to-code scenarios, like the Tufano et al. benchmarks, CodeT5+ performs better on multiple datasets (e.g., achieving 11.36% accuracy on *Android_M*). These findings are in alignment with previous studies, confirming the superiority of these models in program generation tasks [28, 38, 79].

However, a critical observation is that the overall accuracy levels across all models are relatively low, especially in more complex tasks, which raises significant concerns about their reliability in real-world settings. When comparing dataset sizes, it is evident that all models perform better on small-sized datasets than on medium-sized ones. For example, CodeT5+'s accuracy fluctuates markedly from 18.44% on the small *B2F_S* dataset down to just 7.84% on the medium *B2F_M* dataset. Additionally, the performance gap across different datasets is striking, as seen with CodeT5+, which achieves a high of 70.6% on *C#2Java* but decreases to a mere 7.27% on *Google_M*. Such fluctuations and inconsistency in results pose substantial challenges in understanding why models perform so well or poorly on specific datasets. Understanding the reasons is crucial for future research, particularly for improving the reliability and practical application of automated program generation.

> ***Finding 1:*** *Pre-trained encoder-decoder-based models like CodeReviewer and CodeT5+ show superior performance compared to other models across diverse datasets. However, their inconsistent accuracy and significant variability across different tasks and datasets raise concerns about their reliability, emphasizing the need for a deeper understanding of factors affecting model behavior in downstream tasks.*

4.2 RQ2: How Reliable Are Automated Program Generation Approaches?

Reliability and trustworthiness are critically important in automated program generation, especially within the evolving landscape of software engineering [43]. Our preliminary investigation (RQ1) revealed a large disparity in the performance of language models: some models demonstrated exceptionally high performance on certain datasets yet exhibited markedly lower effectiveness on others. This considerable fluctuation raises concerns about the potential overestimation or underestimation of their capabilities due to experimental biases. Considering the potential for both overconfidence and undue skepticism resulting from these inaccuracies, it is important to measure these tools' effectiveness accurately, ensuring their proper application and deployment in software engineering practices. In response to these issues, we undertake an in-depth analysis to uncover potential sources of unrealistic performance evaluation. Our analysis is structured along three different aspects:

- *Data duplication between training and testing sets*: We investigate how similarities or duplications within training and testing datasets might inflate the perceived performance of language models. Such overlaps may create an illusion of high accuracy, masking the true capabilities of these models in novel or diverse scenarios.
- *Data duplication across testing sets*: We investigate the presence of duplicate examples within testing datasets. Duplication within these sets can lead to a misleading evaluation.
- *Output-input similarity analysis*: Finally, we examine the correlation between the outputs generated by the models and their inputs. In automated program generation, the expectation is for models to update or refine input code creatively and accurately. However, when outputs are the same as inputs, it raises questions about the true generative capacity of the models.

In our analysis, we utilize the BLEU score, specifically the *BLEU-4* variant, to systematically evaluate the similarity within the studied datasets. *BLEU-4*, which assesses the co-occurrence of 4-gram sequences, is widely used in prior research [29, 70, 84]. A *BLEU-4* score of 0 corresponds to no similarity, indicating completely unique content, whereas a score of 1 reflects total duplication or exact replication. By applying this metric, we can discern the extent to which our datasets contain unique or duplicative examples, thereby providing an empirical basis for evaluating the potential impact of data duplication on evaluation performance.

4.2.1 Data Duplication between Training and Testing Sets. In automated program generation, the robustness of language model evaluations is crucial. A key threat to this robustness is 'data snooping,'a pitfall where models, due to improper data handling, gain inadvertent access to testing information during training [63]. Such exposure can lead to exaggerated performance metrics, as models may simply recall information rather than apply learned patterns to new data. To prevent this and ensure genuine model generalization, it is essential to assess the overlap between training and testing datasets.

Approach. To determine the similarity score for a test instance t, we compare its source sequence against each instance in the training set $T = \{t_1, t_2, \ldots, t_n\}$. The similarity score, $S(t)$, is the maximum BLEU-4 score obtained from these comparisons:

$$S(t) = \max_{t_i \in T} \text{BLEU-4}(t, t_i). \tag{1}$$

Additionally, we keep track of the index i that yields this maximum score, which can be defined as follows:

$$i_{\max}(t) = \arg\max_{t_i \in T} \text{BLEU-4}(t, t_i). \tag{2}$$

This iterative process is performed for each test instance t, allowing us to quantify the extent of data overlap and identify the potential duplication within the datasets.

Result. Figure 2 and Table 3 present a detailed analysis of the data similarity between training and testing datasets and its influence on model performance. Figure 2 presents the distribution of test data similarity to training data across various datasets and corresponding recalculations of model performance for each similarity range. Furthermore, we have recorded the average similarity score of the output sequences for each test instance with their most similar training instances, as shown in Figure 2.

Our analysis uncovers a wide variation in the similarity scores across an array of datasets, with the notable exception of the *CodeReview* dataset, which stands out due to its lack of highly similar instances. However, datasets such as *Ovirt_M* show that a majority, exceeding 60%, of test data source sequences have a similarity score above 0.8, highlighting a considerable overlap with the training data. In particular, from *Tufano et al.*, comprising *Android_S*, *Android_M*, *Google_S*, *Google_M*, *Ovirt_S*, and *Ovirt_M*, along with the *CodeTrans-Dataset* datasets *Java2C#* and *C#2Java*, we find a significant presence of test samples that are duplicates of the training set (i.e, similarity score = 1). For example, in all *Tufano et al.* datasets, more than 20% of the test samples are identical to those in the training set. These findings suggest the presence of data duplication within the datasets, raising important questions about the possibility of data snooping that could distort the evaluation of model performance.

In Figure 2, we observe an increase in the similarity of target sequences corresponding to an increase in source sequence similarity, up until the source are completely identical ($S(t) = 1$). Taking the *Android_S* dataset as an example, the average similarity for target sequences climbs from 0.1 to approximately 0.6 as the similarity of the source sequences increases. This indicates that target sequences are generally more aligned when the sources are more similar. However, when the

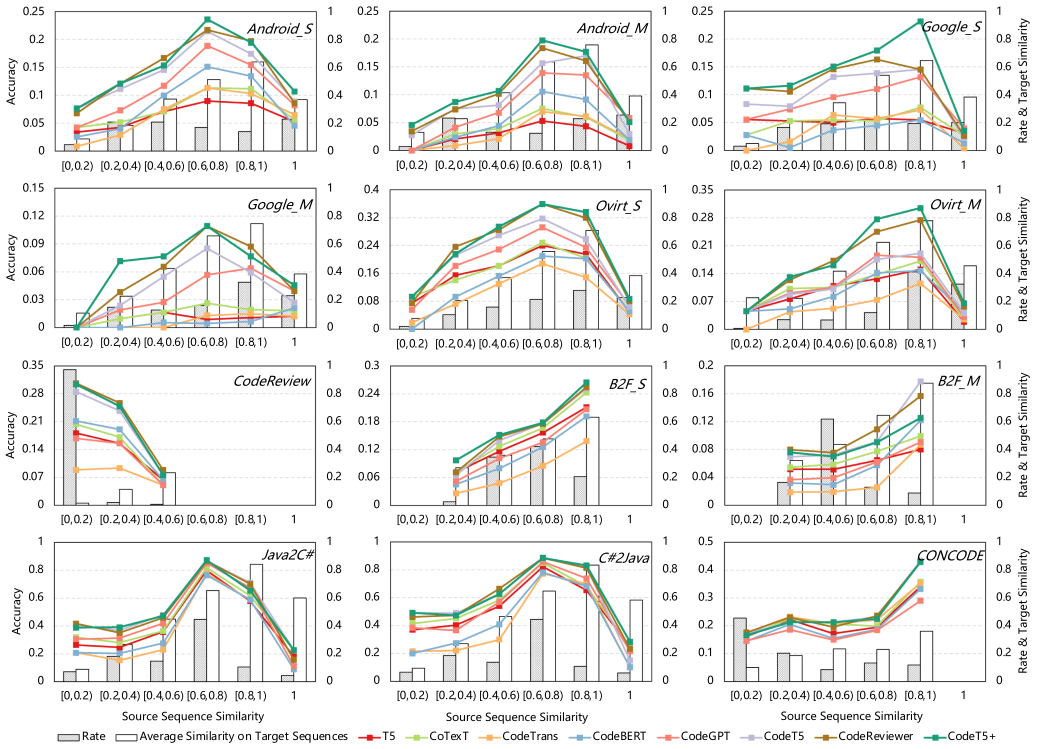

Fig. 2. Distribution of test-training data duplication and model performance across the datasets.

source sequences are identical ($S(t) = 1$), the similarity between target sequences notably drops. For example, in *Android_S*, the average similarity on target sequences drops from 0.6 to 0.4. This observation may indicate a research oversight during dataset preparation, where instances with identical "source + target" pairs are usually removed, but those with identical sources or targets, when considered separately, remain unfiltered.

Figure 2 illustrates that as the similarity between test and training instances increases, so does the performance of language models. For instance, CodeT5+ shows an accuracy of around 10% for test instances with a similarity score below 0.2, which increases to nearly 25% for instances with a similarity score above 0.8 in the *B2F_S* dataset. As shown in Table 2, the average performance of CodeT5+ on the *B2F_S* dataset is 18.44%. This suggests that models may be leveraging memorized patterns from the training data rather than demonstrating true generalization capabilities. Consequently, the ability of language models to handle unseen or novel test instances remains a considerable challenge for future research. A notable drop in model performance is also observed when the source sequences are exactly the same, a decrease that corresponds with the fall in similarity among output sequences. This could be due to the common practice during dataset preparation where instances of identical "source + target" pairs are removed, possibly leading to an oversight of singular duplicates in sources or targets.

Table 3 further describes the impact on the performance of models such as CodeReviewer and CodeT5+ when instances with high similarity scores are excluded from the test sets. Whereas the *CodeReview* dataset shows no change due to a lack of closely matched samples, other datasets usually display a marked decrease in accuracy once we remove test cases with high similarity

Table 3. Model Performance Before and After Removing High-Similarity Test Instances

		Android_S	Android_M	Google_S	Google_M	Ovirt_S	Ovirt_M	CodeReview	B2F_S	B2F_M	Java2C#	C#2Java	CONCODE
Test Samples Percentage (>0.6)		53.69%	60.62%	60.88%	71.21%	71.72%	85.74%	0.05%	62.81%	21.82%	59.80%	61.20%	25.25%
CodeReviewer	Original Accuracy	14.68%	10.40%	11.81%	6.85%	25.49%	18.18%	30.43%	17.94%	8.77%	63.10%	70.40%	22.65%
	New Accuracy	13.61%	8.02%	12.61%	4.81%	25.25%	14.39%	30.44%	14.24%	7.64%	40.30%	53.87%	19.26%
	Original BLEU	0.70	0.72	0.71	0.73	0.75	0.77	0.86	0.75	0.85	0.92	0.93	0.59
	New BLEU	0.70	0.72	0.70	0.67	0.73	0.72	0.86	0.76	0.85	0.89	0.91	0.56
CodeT5+	Original Accuracy	15.20%	11.36%	14.27%	7.27%	26.06%	20.03%	30.12%	18.44%	7.84%	63.90%	70.60%	21.85%
	New Accuracy	13.09%	9.07%	13.29%	6.97%	25.13%	14.21%	30.14%	14.75%	7.11%	42.04%	53.09%	18.39%
	Original BLEU	0.70	0.72	0.72	0.72	0.75	0.77	0.85	0.75	0.85	0.93	0.93	0.59
	New BLEU	0.70	0.72	0.71	0.66	0.74	0.71	0.85	0.76	0.85	0.89	0.91	0.56

scores. In the *Android_S* dataset, for example, the accuracy of *CodeT5+* decreases from 15.2% to 13.09%. In some cases, the drop is even more apparent; *CodeT5+* falls from 70.6% to 53.09% on the *C#2Java* dataset. This highlights the critical role that a varied and comprehensive test set plays in ensuring the accuracy of a model's performance evaluation.

Finding 2: *Our results reveal that multiple program generation datasets contain substantial duplications between their training and testing sets. The increased similarity between these sets typically leads to exaggerated performance metrics, raising concerns about the models' generalization capabilities and suggesting potential flaws in data handling.*

4.2.2 Data Duplication across Testing Sets. Building upon the test-training analysis, this section examines intra-set duplications within the test datasets. The presence of duplicated instances in test sets can lead to unreliable evaluation results. If such duplications go unaddressed, the model's efficacy may end up being evaluated on a narrow subset of repeated test instances rather than a diverse range of samples. To address this concern, we investigate the prevalence of data duplication within the testing sets based on our studied datasets.

Approach. For each test instance t, we investigate potential duplication within the test set by comparing its source sequence with that of every other test instance. The maximum BLEU-4 score from these comparisons is recorded, along with the index of the test instance that yields this maximum score:

$$D(t) = \max_{\substack{t' \in \mathbf{T}_{\text{test}} \\ t' \neq t}} \text{BLEU-4}(t, t'), \tag{3}$$

$$j_{\max}(t) = \arg\max_{\substack{t' \in \mathbf{T}_{\text{test}} \\ t' \neq t}} \text{BLEU-4}(t, t'). \tag{4}$$

Here, $D(t)$ is the similarity score for the test instance t within testing data, and $j_{\max}(t)$ is the index of another test instance t' that is most similar to t. This process is iteratively performed for each test instance in the dataset, allowing us to identify exact and near-duplicates within the test set.

Result. Figure 3 shows the distribution of similarity within the test data of our studied datasets. We observe that except for the *B2F_S* and *B2F_M* datasets, which exhibit no duplicate instances, a considerable number of datasets include duplicates. For example, *Android_S* records 11% of its test instances as duplicates based on source sequences, and similarly, *Ovirt_M* possesses more than 10% duplication. In line with earlier observations from Figure 2, the average similarity scores of the target sequences for these duplicate instances are substantially lower compared to the dataset at large. This indicates that while testing language model performance on program generation, identical source sequences are often provided, but the models are expected to provide different outputs. Figure 4 provides an example from the *Android_S* dataset where two test instances with the same source code require different correct outputs from the model. This practice leads to an inherently unfair evaluation scenario, where the same test instance is associated with different

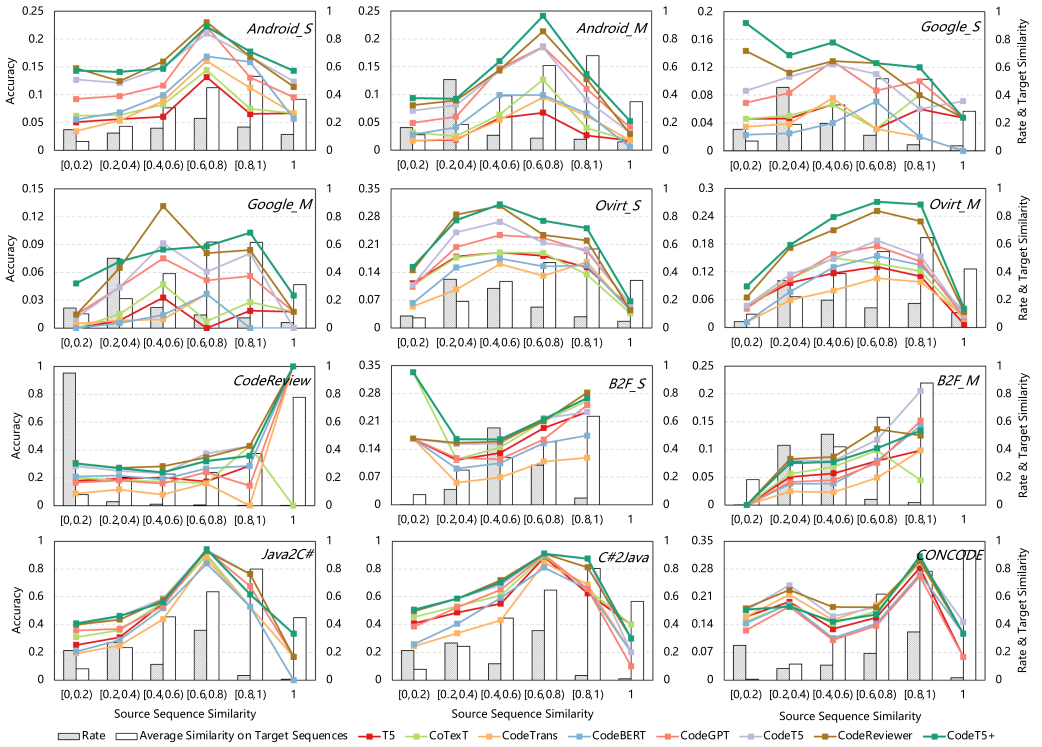

Fig. 3. Distribution of data duplication within testing sets and corresponding model performance metrics.

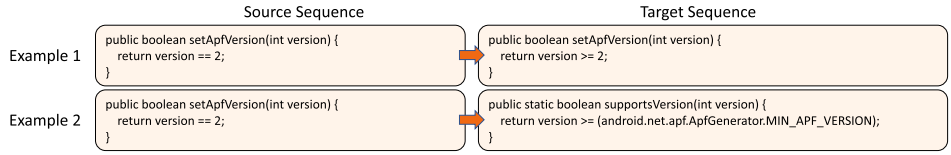

Fig. 4. Examples of test instances with duplicated sources and different targets from *Android_S*.

performance expectations. Figure 3 indicates that performance on these duplicated test instances can vary greatly from the average, potentially giving an inaccurate representation of model performance. Our findings emphasize the need to carefully construct test sets, avoiding the pitfalls of duplications that can compromise reliable model performance evaluation. It is crucial to construct detailed and impartial evaluation approaches that truly measure the language models' ability to generate code across a wide array of test scenarios.

> ***Finding 3:*** *In our examination of 12 datasets, we find that 10 contain duplicated source sequences within their test instances, despite requiring models to generate different targets. Such inconsistencies in test design lead to evaluations that may not accurately reflect the true capabilities of program generation approaches, thus compromising the reliability of performance evaluation.*

4.2.3 Output-Input Similarity Analysis. The goal of automated program generation is to refine or create new code sequences that accurately address specified functionality changes or

Table 4. Comparison of Output-Input Duplication Rates across Language Models for Program Generation Tasks

	Android_S	Android_M	Google_S	Google_M	Ovirt_S	Ovirt_M	CodeReview	B2F_S	B2F_M	Java2C#	C#2Java	CONCODE
Source vs Target	0%	0%	0%	0%	0%	0%	1%	0%	0%	2%	2%	0%
T5	78%	80%	83%	81%	53%	61%	37%	21%	71%	2%	2%	0%
CoTexT	74%	73%	80%	77%	54%	58%	33%	22%	73%	2%	2%	0%
CodeTrans	36%	36%	30%	27%	36%	41%	50%	71%	84%	2%	2%	0%
CodeBERT	21%	27%	10%	5%	18%	29%	36%	51%	78%	1%	1%	0%
CodeGPT	36%	47%	56%	57%	33%	31%	41%	31%	77%	2%	2%	0%
CodeT5	40%	44%	60%	51%	27%	46%	20%	24%	64%	2%	2%	0%
CodeReviewer	15%	31%	26%	37%	10%	20%	14%	11%	44%	2%	2%	0%
CodeT5+	21%	29%	31%	32%	16%	18%	17%	10%	35%	2%	2%	0%

requirements. In most prior works, model predictions are compared against target outputs to calculate accuracy and other performance metrics. We extend this evaluation to compare the generated outputs with the original inputs, assessing whether the models are merely mirroring the inputs or actually generating updated code sequences.

Approach. Different from our earlier focus on similarity scores within and across test sets, here we focus specifically on the percentage of identical sequences in the model outputs as compared to their source code sequences. For this comparison, we utilize a direct string comparison method, discounting special tokens (e.g., "<pad>", "<s>", and "</s>"), which are typically used for formatting and do not contribute to the functional content of the code. Through this process, for each model's predictions, we calculate the proportion of outputs that are exactly the same as the source sequences.

Result. Table 4 presents the rates of duplication between model outputs and inputs across several language models on the studied datasets. The table reveals a considerable variance in the rates across different datasets and models. Notably, only the *CodeReview*, *Java2C#*, and *C#2Java* datasets display minimal duplication between source and target sequences, with rates as low as 1% to 2%, suggesting that these datasets effectively evaluate the models' capacity for generating new code. When comparing model outputs with source code sequences, language models present high rates of output-input duplication. For example, the rate of T5 on *Android_S* and *Android_M* reaches 78% and 80%, respectively. Such a high duplication rate suggests that a substantial portion of the output code is replicated from the inputs, calling into question the models' generative capabilities. Although models like CodeReviewer and CodeT5+ show superior performance, as indicated in RQ1, they still exhibit a significant degree of output duplication with their inputs (e.g., 35% for CodeT5+ on the *B2F_S* dataset and 32% on *Google_M*). The analysis of the *CodeReview* dataset, even within the context of code+comment-to-code tasks, reveals a significant level of duplication between the source code sequences and the models' outputs. Note that the *CONCODE* dataset, which focuses on text-to-code tasks, demonstrates zero duplication, indicating the models' potential to generate entirely new code from textual prompts. In the context of language models applied to code translation tasks within the *Java2C#* and *C#2Java* datasets, we observe a commendably low duplication rate. The observed variability in duplication rates across datasets and tasks underscores the imperative for nuanced and robust evaluation metrics that can accurately reflect the true generative capabilities of language models.

> ***Finding 4:*** *In code review and code repair tasks, it is observed that language models frequently generate outputs that are identical to the input sequences, presenting a potential limitation in their ability to generate novel code solutions.*

4.3 RQ3: Can We Explain Why Automated Program Generation Approaches Can (or Fail to) Generate Code Sequences Reliably?

Most pre-trained language models operate as black-box systems, obscuring their internal decision-making processes. As highlighted in RQ1, the accuracy of these models with beam search set at 1, is not particularly high, underscoring the uncertainty in their output reliability. Furthermore, the results from RQ2 suggest that the evaluation of these models may be compromised by certain impractical experimental settings. Consequently, it is necessary to employ explainable AI approaches to demystify the internal workings of these models. In this section, we employ gradient-based SHAP to examine and understand the decision-making processes of automated program generation models. This approach aims to uncover the factors behind a model's ability or failure to generate accurate and reliable code sequences. To ensure a focused and relevant analysis, we limit our examination to the models demonstrating superior performance in previous sections (i.e., CodeT5, CodeReviewer, and CodeT5+). This focused examination is designed to gain insights into what contributes to their effective and reliable performance in program generation, with the aim of guiding future developments in this field.

4.3.1 Understanding Token Importance. In this section, we delve into the learning patterns of language models concerning program generation by examining the average feature importance of different types of tokens. These tokens represent the fundamental building blocks of programming languages and are crucial for understanding the model's focus during the program generation process. We analyze five types of tokens, including the following:

— *Identifiers* are unique names for variables, classes, methods, and so on.
— *Keywords* refer to programming language-specific words with pre-determined standard meanings.
— *Operators* are special symbols that represent some operation on data.
— *Separators* are special symbols used to indicate the group of code.
— *Literals* refer to constant values.

Result. Figure 5 presents the average feature importance of token types across various datasets when analyzed through the CodeT5, CodeReviewer, and CodeT5+ models. The bar chart provides a comparative visualization of how each model weighs the significance of identifiers, keywords, operators, separators, and literals during the program generation process. It is apparent across the models: identifiers and keywords tend to be assigned higher importance scores, showing their critical role in understanding the syntactic and semantic structure of the code. This suggests that models are possibly prioritizing the recognition of variable names, function calls, and control structures, which are key to the functionality of the code. Operators and separators, while varied across models and datasets, generally exhibit moderate importance. This reflects a slight comprehension by the models of the operational logic and structural delineation within the code, which, although less emphasized than identifiers and keywords, are still recognized as essential components of program logic.

> ***Finding 5:*** *The explanation results reveal that identifiers and keywords are consistently assigned higher importance scores compared to operators and separators, indicating that language models prioritize syntactic and semantic understanding in program generation.*

4.3.2 Model Performance Under Input Token Reduction. In this section, we investigate the robustness of program generation models when faced with reduced input tokens. Drawing from the

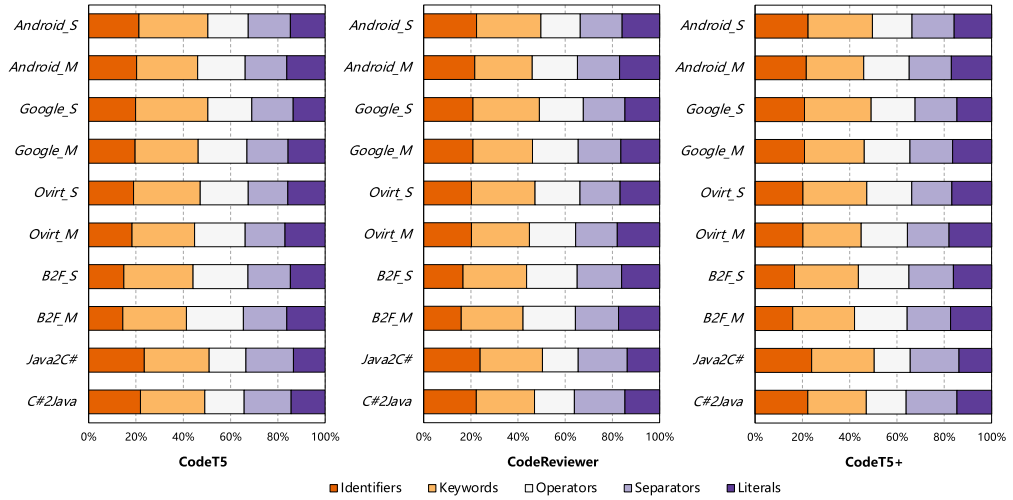

Fig. 5. Average feature importance of token types highlighted by language models on the studied datasets.[2]

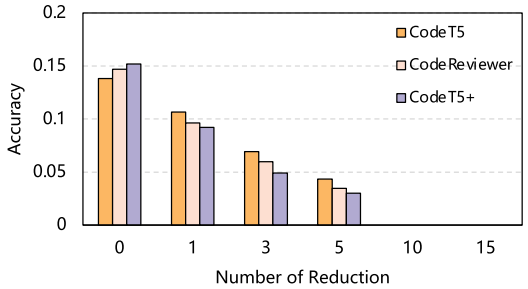

Fig. 6. Impacts of token reduction size on *Android_S*.

explanation results, we investigate how the selective deletion of tokens classified as less important impacts the models' ability (i.e., lowest feature importance) to generate accurate code sequences.

Impacts of Token Reduction Size. In this study, we examine the impact of selectively removing tokens from the input sequences based on their feature importance scores. For each test case, tokens are ranked by their importance, and we methodically remove a specified number of the least important tokens, with placeholders inserted to preserve the format of the code. This process generates new "source-target" pairs, which are then fed back into the models. The experiment is conducted with incremental token removals, specifically at counts of 1, 3, 5, 10, and 15, to understand how the absence of certain tokens affects the models' code generation capabilities. In our analysis, we denote the original "source-target" testing instances with a 0 to differentiate them from the modified instances.

Result. The results presented in Figure 6, which focus on the *Android_S* dataset, provide a clear illustration of the performance degradation associated with the incremental removal of the least important tokens. For example, in the CodeT5+ model, eliminating just one token leads to a significant drop in performance, from 15.2% to 9.19%. As the number of tokens removed increases to 5,

[2]Note that the datasets *CodeReview* and *CONCODE* are excluded since it is challenging to distinguish the token types from the source sequences due to the abundance of comments and natural language descriptions.

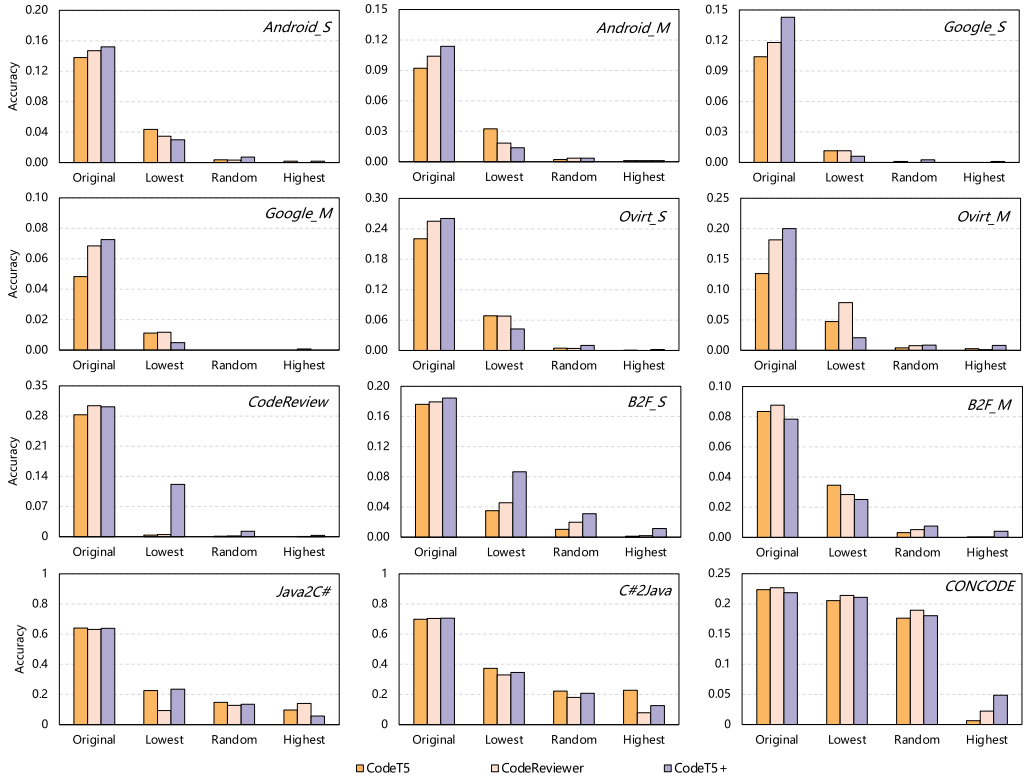

Fig. 7. Performance upon input token reduction using different strategies.

the performance further declines to a mere 3%. Beyond the removal of 10 tokens, the model's performance drops to zero, indicating a complete inability to generate the correct code sequence. This pattern is not unique to CodeT5+; similar trends are observed across all three studied models. The significant decrease in performance, even with the removal of tokens previously considered less important, suggests a complex interdependence among the various input features. These findings raise important questions about the robustness of program generation models, particularly their sensitivity to changes in input and their ability to adapt and maintain accuracy under modified conditions.

Comparative Analysis of Token Removal Strategies. To evaluate the reliability of the explainable AI approach, we compare strategies to remove tokens from the input sequences, specifically targeting the lowest and highest importance tokens identified by our model, and also using a random token removal for comparison. This comparison is designed to investigate how each removal strategy affects the model's performance. By comparing the results of targeted versus random token removal, we aim to examine the accuracy of the explainable AI approach's ability to identify important features for program generation. In each of these three strategies, we consistently remove a set number of five tokens for a standardized comparison.

Result. As shown in Figure 7, the removal of tokens demonstrates a clear impact on performance across different datasets. Specifically, removing tokens identified with the lowest importance typically leads to a smaller decline in performance compared to either random removal or the removal of the most important tokens. For example, within the *Android_S* dataset, the accuracy of

the CodeT5+ model falls to 3% after removing the least important tokens but drops almost to zero with random or most important token removal. Similarly, within the *B2F_S* and *CONCODE* dataset, the accuracy of the CodeT5+ model on lowest and random removal is much higher compared to removing the most important tokens. Thus, these findings suggest that explainable AI approaches can help us determine the significance of different input tokens for program generation. Moreover, the consistent decrease in performance across various datasets, following the removal of important tokens, presents a lack of robustness in these language models.

> ***Finding 6:*** *Our results show that explainable AI methods can effectively identify feature importance when generating code sequences. The substantial performance decrease observed when crucial tokens are removed underlines a vulnerability in language models, showing a need for enhanced model robustness.*

5 DISCUSSION

In this study, we have identified several interesting findings of the reliability and explainability of automated program generation approaches. We now discuss the main implications and limitations of our study.

5.1 Implications

Reliability. High reliability is essential for the real-world usage of language model-based automated program generation systems. As highlighted by prior works [62, 83], state-of-the-art language models are vulnerable to adversarial attacks and can be fooled into recommending wrong code. This finding indicates that the existing language models still suffer from reliability issues. However, most prior works pay more attention to improving the accuracy of automated program generation systems, but they neglect to evaluate whether the proposed methodologies are sufficiently reliable.

Our study first replicates the good performance presented in previous research. Our results further indicate that data duplication commonly exists in existing state-of-the-art program generation datasets (i.e., duplication between training and testing sets, as well as duplication within testing sets), and the issues provide unreliable or unrealistic performance evaluation in prior research. Further, our results show that pre-trained language models frequently generate outputs replicated from the inputs, outputting numerous unchanged code sequences. These findings provide evidence that language model based program generation approaches suffer from serious reliability threats since the performance is overestimated or underestimated by unreliable experimental analysis. This not only provides unreliable evaluation performance of deep learning systems but also raises concerns regarding their deployment in real-world applications. Consequently, there is a need for research on enhancing the quality and reliability of automated program generation research, which can further benefit the deployment of deep learning systems in the real world. First, more research efforts should focus on the dataset quality. Except for the data duplication issues [5], Shi et al. [64] and Sun et al. [65] have demonstrated that data noise was prevalent in widely used benchmark datasets in code summarization and code search. Therefore, for future research, it is important to construct a rigorous evaluation methodology supported by reliable and standardized benchmarks. In addition, instead of only relying on accuracy, more comprehensive evaluation metrics should be employed to provide a reliable evaluation for language models.

Explainability. Pre-trained models are black-box models. Thus, prior research usually disregards understanding why the language model based program generation approaches make a specific prediction. Our study employs a model-agnostic explainable AI approach to explain language models.

Furthermore, our results reveal several insightful findings that can inspire future research. First, we observe that language model based program generation models pay much more attention to keywords and identifiers of programming language compared with operators and separators. This indicates that language models are capable of recognizing code grammar and structural information. However, our findings reveal a significant decline in the performance of language models with the removal of even a few tokens, including those considered least important, highlighting a lack of robustness in these models. These observations from explanation results help us better understand inference behaviors and learning abilities of program generation behaviors. Consequently, our study also proves that explainable AI approaches are an effective and promising approach to analyzing or improving the reliability of language model based automated program generation systems. More research can be initiated on this aspect.

5.2 Threats to Validity

The primary threat to internal validity mainly lies in the model architecture and hyper-parameter setting. We use eight program generation models, which are based on the same model settings in the original papers. It is expected that hyper-parameter tuning would bring performance improvements. Instead, the goal of our work is not to find the best setting, but to fairly investigate the reliability and explainability of program generation models. The external threats to validity mainly lie in the studied datasets and the generalizability of the results. In this study, we used five different types of program generation datasets. We considered four types of code generation tasks (i.e., code review, code repair, code translation, and code generation), different token sizes (i.e., small and medium), different programming languages (e.g., Java, Python, C#), and three types of inputs (i.g., only code, code+comments, only text). For reproducibility purposes, we provided a replication package to facilitate future work to replicate our approach on more repositories and tasks.

6 RELATED WORK

6.1 Language Models for Program Generation

In recent years, researchers have increasingly applied language models, including pre-trained models, to program generation tasks. Tufano et al. [72] developed a deep learning approach using RNNs to automatically transform source code in the code review context. Thongtanunam et al. [70] further introduced advanced Transformer architecture and a Byte-Pair Encoding (BPE) approach to handle the out-of-vocabulary and long sequence problems. To better learn code properties, pre-training techniques are increasingly adopted in the code review scenario [12, 28, 38, 74, 87]. Hong et al. [28] proposed a CodeT5-based approach to recommend code review comments automatically. Li et al. [38] developed a Transformer-based encoder-decoder model that is pre-trained on large code reviewer-specific data for code refinement tasks. Many studies [11, 35] involve translating code from one programming language to another.

In contrast to previous research, this article undertakes an empirical study to evaluate how reliable language models are in program generation. Most prior studies demonstrate that the proposed approach is quite accurate. However, these studies do not provide convincing explanations for the superior accuracy of their approach. Our study focuses on understanding the mechanisms behind language models in program generation scenarios and attempting to analyze potential issues concealed by decent performance.

6.2 Explainable Language Model Based Program Generation

Previous research has already investigated applying explainable AI approaches, despite not being well suited, to automated program generation [21, 32, 41, 68, 69]. There have been many works

that explore the attention mechanism to explain language models of code [3, 49, 55, 76, 88]. Zhang et al. [88] employed attention mechanisms to dig into critical statements and tokens learned by pre-trained language models in code search and code summarization. Paltenghi and Pradel [55] explored to what extent the attention weights of language models match the reasoning of skilled humans in code summarization. Wan et al. [76] analyzed the self-attention weights of pre-trained models of code and discovered that attention could capture high-level source code structural information. Besides the attention mechanism, various explainable AI approaches have also been employed to explain NMT models of source code. For example, Cito et al. [15] integrated counterfactual explanation techniques for language models that predict certain properties of code or code changes. Rabin et al. [60] provided a model-agnostic approach to identify critical input features for models of code and demonstrated that the approach enables code simplification in code search and variable misuse debugging.

Prior work has mainly focused on explaining code-to-text or text-to-code tasks (e.g., code search and code summarization). Our study aims to understand the programming language models for program generation (i.e., code-to-code). In addition, previous works capture structural knowledge learned by language models, whereas we dig more into model behavior understanding in various experimental scenarios.

6.3 Robustness of Language Models for Source Code

Owing to the increasing research interest in this pattern, there are many recent studies proposed to explore the robustness and effectiveness of source code models. A plethora of studies have demonstrated that source code models are vulnerable to adversarial attacks [51, 62, 83, 89]. For example, Yang et al. [83] developed ALERT, a black-box attack approach that adversarially modifies code snippets to force pre-trained code models to produce incorrect outputs. Schuster et al. [62] showed that code completion models are susceptible to poisoning attacks by adding some carefully crafted files to the training data of a model. Some empirical studies have been conducted to empirically investigate the performance of programming language models [13, 47, 77, 84]. For example, Zeng et al. [84] suggested that developing an almighty pre-trained code model across task types is challenging, and more rigorous evaluations are required.

Distinct from previous work, we empirically examine the robustness and limitations of pre-trained code models on automated program generation. Furthermore, we apply explainable AI approaches to assist us in understanding why code models are not robust enough for automated language generation, which is currently rarely explored in software engineering.

6.4 Reliability in Language Models for Source Code

With the increasing application of advanced large language models in software engineering, ensuring their trustworthiness becomes important [42, 43]. She et al. [63] reviewed common experimental biases in language models for code research, such as data noise, labeling errors, and inappropriate evaluation approaches. Allamanis [5] investigated the effects of code duplication and found that performance metrics could be inflated by up to 100% when testing on duplicated code corpora, as opposed to de-duplicated corpora which more accurately reflect real-world usage by software engineers. Nie et al. [52] explored labeling errors in vulnerability detection datasets, noting that incorrectly labeling a non-vulnerable sample as vulnerable was a more common issue. Additionally, Nong et al. [53] highlighted that datasets like SARD [6] often contain synthetic examples that are not representative of real-world code, characterized by smaller vocabulary, shorter program lengths, and higher pattern frequency. As language models for code begin to be deployed in real-world applications (e.g., GitHub Copilot [26] and ChatGPT [54]), new challenges emerge, such as real-world constraints, security concerns, and vulnerability to attacks [16, 40, 46, 85].

Different from prior studies, our research specifically focuses on reliability issues in program generation. We comprehensively examine five datasets to assess the impacts arising from the dataset, evaluation approaches, and the inherent robustness of models in various program generation contexts. This approach allows us to provide deeper insights into how these factors collectively influence the reliability and performance of language models in generating code.

7 CONCLUSION

In this work, we investigated the reliability and explainability of language models to perform automated program generation. Our study underlined the following practical conclusion:

- The performances and key findings of prior works in automated program generation can be duplicated.
- Automated program generation models are not reliable. Prior works suffer from serious reliability concerns, resulting in unrealistic performance evaluation.
- Explainable AI approaches are a promising approach to help us understand program generation models. We discover that program generation models capture distinct patterns, but identifiers and keywords are consistently assigned higher importance scores.

Further, our study highlighted two meaningful insights. First, our study demonstrates that prior program generation approaches do not have perfect robustness and reliability, pointing to opportunities for future improvement. However, explainable AI approaches help us better understand model inference behaviors.

REFERENCES

[1] Google. 2021. Google BigQuery. Retrieved September 2, 2022 from https://console.cloud.google.com/marketplace/details/github/github-repos

[2] Gerrit Code Review. 2022. Gerrit Code Review Home Page. Retrieved September 2, 2022 from https://www.gerritcodereview.com/

[3] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2655–2668.

[4] J. Alammar. 2021. Ecco: An open source library for the explainability of transformer language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*.

[5] Miltiadis Allamanis. 2019. The adverse effects of code duplication in machine learning models of code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153.

[6] Paul E. Black et al. 2017. SARD: A software assurance reference dataset. In *Anonymous Cybersecurity Innovation Forum*. https://samate.nist.gov/SARD/

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.

[8] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar Devanbu, and Baishakhi Ray. 2022. NatGen: Generative pre-training by "naturalizing" source code. *arXiv preprint arXiv:2206.07585* (2022).

[9] Saikat Chakraborty and Baishakhi Ray. 2021. On multi-modal learning of editing source code. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*. IEEE, 443–455.

[10] Chi Chen, Xin Peng, Zhenchang Xing, Jun Sun, Xin Wang, Yifan Zhao, and Wenyun Zhao. 2022. Holistic combination of structural and textual code information for context based API recommendation. *IEEE Transactions on Software Engineering* 48, 8 (2022), 2987–3009.

[11] Xinyun Chen, Chang Liu, and Dawn Song. 2018. Tree-to-tree neural networks for program translation. *Advances in Neural Information Processing Systems* 31 (2018), 1–11.

[12] Nadezhda Chirkova and Sergey Troshin. 2021. Empirical study of transformers for source code. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 703–715.

[13] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, and Gabriele Bavota. 2022. An empirical study on the usage of transformer models for code completion. *IEEE Transactions on Software Engineering* 48, 12 (2022), 4818–4837. https://doi.org/10.1109/TSE.2021.3128234

[14] Jürgen Cito, Satish Chandra, Chakkrit Tantithamthavorn, and Hadi Hemmati. 2023. Expert perspectives on explainability. *IEEE Software* 40, 3 (2023), 84–88. https://doi.org/10.1109/MS.2023.3255663

[15] Jürgen Cito, Isil Dillig, Vijayaraghavan Murali, and Satish Chandra. 2022. Counterfactual explanations for models of code. In *Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '22)*. IEEE, 125–134.

[16] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming Jack Jiang. 2023. GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.

[17] Ahmed Elnaggar, Wei Ding, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Silvia Severini, Florian Matthes, and Burkhard Rost. 2021. CodeTrans: Towards cracking the language of silicon's code through self-supervised deep learning and high performance computing. *arXiv preprint arXiv:2104.02443* (2021).

[18] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. *arXiv preprint arXiv:2310.03533* (2023).

[19] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 1536–1547.

[20] Michael Fu, Van Nguyen, Chakkrit Tantithamthavorn, Dinh Phung, and Trung Le. 2023. Vision Transformer-inspired automated vulnerability repair. *ACM Transactions on Software Engineering and Methodology*. Published Online, November 13, 2023. https://doi.org/10.1145/3632746

[21] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th IEEE/ACM International Conference on Mining Software Repositories (MSR '22)*. ACM, 608–620. https://doi.org/10.1145/3524842.3528452

[22] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Yuki Kume, Van Nguyen, Dinh Phung, and John Grundy. 2023. AIBugHunter: A practical tool for predicting, classifying and repairing software vulnerabilities. *Empirical Software Engineering*. Published Online, November 20, 2023.

[23] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: A T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. 935–947.

[24] Michael Fu, Chakkrit Tantithamthavorn, Van Nguyen, and Trung Le. 2023. ChatGPT for vulnerability detection, classification, and repair: How far are we? In *Proceedings of the 30th Asia-Pacific Software Engineering Conference (APSEC '23)*.

[25] Michael Fu, Chakkrit Tantithamthavorn Van Nguyen, Trung Le, and Dinh Phung. 2023. VulExplainer: A transformer-based hierarchical distillation for explaining vulnerability types. *IEEE Transactions on Software Engineering* 49, 10 (2023), 4550–4565.

[26] GitHub. 2023. GitHub Copilot. Retrieved January 20, 2024 from https://copilot.github.com

[27] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE '18)*. IEEE, 933–944.

[28] Yang Hong, Chakkrit Tantithamthavorn, Patanamon Thongtanunam, and Aldeida Aleti. 2022. CommentFinder: A simpler, faster, more accurate code review comments recommendation. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. 507–519.

[29] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large language models for software engineering: A systematic literature review. *arXiv preprint arXiv:2308.10620* (2023).

[30] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[31] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. *arXiv preprint arXiv:1808.09588* (2018).

[32] Jirayus Jiarpakdee, Chakkrit Tantithamthavorn, Hoa Khanh Dam, and John C. Grundy. 2022. An empirical study of model-agnostic techniques for defect prediction models. *IEEE Transactions on Software Engineering* 48, 2 (2022), 166–185. https://doi.org/10.1109/TSE.2020.2982385

[33] Jirayus Jiarpakdee, Chakkrit Tantithamthavorn, and John C. Grundy. 2021. Practitioners' perceptions of the goals and visual explanations of defect prediction models. In *Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR '21)*. IEEE, 432–443. https://doi.org/10.1109/MSR52588.2021.00055

[34] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. 2020. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896* (2020).

[35] Marie-Anne Lachaux, Baptiste Roziere, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *arXiv preprint arXiv:2006.03511* (2020).

[36] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. *arXiv preprint arXiv:2207.01780* (2022).

[37] Yaoxian Li, Shiyi Qi, Cuiyun Gao, Yun Peng, David Lo, Zenglin Xu, and Michael R. Lyu. 2022. A closer look into transformer-based code intelligence through code transformation: Challenges and opportunities. *arXiv preprint arXiv:2207.04285* (2022).

[38] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, and Neel Sundaresan. 2022. CodeReviewer: Pre-training for automating code review activities. *arXiv preprint arXiv:2203.09095* (2022).

[39] Fang Liu, Ge Li, Yunfei Zhao, and Zhi Jin. 2020. Multi-task learning based pre-trained language model for code completion. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 473–485.

[40] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2023. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *arXiv preprint arXiv:2307.12596* (2023).

[41] Yue Liu, Chakkrit Tantithamthavorn, Li Li, and Yepang Liu. 2022. Explainable AI for Android malware detection: Towards understanding why the models perform so well? In *Proceedings of the IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE '22)*. IEEE, 169–180. https://doi.org/10.1109/ISSRE55969.2022.00026

[42] Yue Liu, Chakkrit Tantithamthavorn, Yonghui Liu, Patanamon Thongtanunam, and Li Li. 2022. AutoUpdate: Automatically recommend code updates for Android apps. *arXiv preprint arXiv:2209.07048* (2022).

[43] David Lo. 2023. Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps. *arXiv preprint arXiv:2309.04142* (2023).

[44] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.

[45] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS '17)*. 4768–4777.

[46] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. On the robustness of code generation techniques: An empirical study on GitHub Copilot. *arXiv preprint arXiv:2302.00438* (2023).

[47] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*. IEEE, 336–347.

[48] Ahmad Haji Mohammadkhani, Nitin Sai Bommi, Mariem Daboussi, Onkar Sabnis, Chakkrit Tantithamthavorn, and Hadi Hemmati. 2023. A systematic literature review of explainable AI for software engineering. *arXiv preprint arXiv:2302.06065* (2023).

[49] Ahmad Haji Mohammadkhani, Hadi Hemmati, and Chakkrit Tantithamthavorn. 2023. Explaining transformer-based code models: What do they learn? When they do not work? In *Proceedings of the 23rd IEEE International Working Conference on Source Code Analysis and Manipulation*. IEEE.

[50] Anh Tuan Nguyen, Tung Thanh Nguyen, and Tien N. Nguyen. 2015. Divide-and-conquer approach for multi-phase statistical migration for source code (T). In *Proceedings of the 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE '15)*. IEEE, 585–596.

[51] Phuong T. Nguyen, Claudio Di Sipio, Juri Di Rocco, Massimiliano Di Penta, and Davide Di Ruscio. 2021. Adversarial attacks to API recommender systems: Time to wake up and smell the coffee? In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*. IEEE, 253–265.

[52] Xu Nie, Ningke Li, Kailong Wang, Shangguang Wang, Xiapu Luo, and Haoyu Wang. 2023. Understanding and tackling label errors in deep learning-based vulnerability detection (experience paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 52–63.

[53] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai. 2022. Generating realistic vulnerabilities via neural code editing: An empirical study. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 1097–1109. https://doi.org/10.1145/3540250.3549128

[54] OpenAI. 2023. GPT-4 technical report. arXiv: 2303.08774 [cs.CL] (2023).

[55] Matteo Paltenghi and Michael Pradel. 2021. Thinking like a developer? Comparing the attention of humans with neural models of code. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*. IEEE, 867–879.

[56] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. 311–318.

[57] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32 (2019), 1–12.

[58] Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Annibal, Alec Peltekian, and Yanfang Ye. 2021. CoTexT: Multi-task learning with code-text transformer. In *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog '21)*. 40–47.

[59] Chanathip Pornprasit, Chakkrit Tantithamthavorn, Patanamon Thongtanunam, and Chunyang Chen. 2023. D-ACT: Towards diff-aware code transformation for code review under a time-wise evaluation. In *Proceedings of the International Conference on Software Analysis, Evolution, and Reengineering (SANER '23)*. ACM. https://doi.org/10.1109/SANER56733.2023.00036

[60] Md. Rafiqul Islam Rabin, Vincent J. Hellendoorn, and Mohammad Amin Alipour. 2021. Understanding neural code intelligence through program simplification. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 441–452.

[61] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.

[62] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. 2021. You autocomplete me: Poisoning vulnerabilities in neural code completion. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*. 1559–1575.

[63] Xinyu She, Yue Liu, Yanjie Zhao, Yiling He, Li Li, Chakkrit Tantithamthavorn, Zhan Qin, and Haoyu Wang. 2023. Pitfalls in language models for code intelligence: A taxonomy and survey. *arXiv:2310.17903 [cs.SE]* (2023).

[64] Lin Shi, Fangwen Mu, Xiao Chen, Song Wang, Junjie Wang, Ye Yang, Ge Li, Xin Xia, and Qing Wang. 2022. Are we building on the rock? On the importance of data preprocessing for code summarization. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 107–119.

[65] Zhensu Sun, Li Li, Yan Liu, Xiaoning Du, and Li Li. 2022. On the importance of building high-quality training datasets for neural code search. In *Proceedings of the 44th International Conference on Software Engineering*. 1609–1620.

[66] Wannita Takerngsaksiri, Chakkrit Tantithamthavorn, and Yuan-Fang Li. 2024. Syntax-aware on-the-fly code completion. *Information and Software Technology* 165 (2024), 107336.

[67] Chakkrit Tantithamthavorn, Jürgen Cito, Hadi Hemmati, and Satish Chandra. 2023. Explainable AI for SE: Challenges and future directions. *IEEE Software* 40, 3 (2023), 29–33. https://doi.org/10.1109/MS.2023.3246686

[68] Chakkrit Tantithamthavorn and Jirayus Jiarpakdee. 2021. Explainable AI for software engineering. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*. IEEE, 1–2. https://doi.org/10.1109/ASE51524.2021.9678580

[69] Chakkrit Tantithamthavorn, Jirayus Jiarpakdee, and John Grundy. 2021. Actionable analytics: Stop telling me what it is; please tell me what to do. *IEEE Software* 38, 4 (2021), 115–120. https://doi.org/10.1109/MS.2021.3072088

[70] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. AutoTransform: Automated code transformation to support modern code review process. In *Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. 237–248. https://doi.org/10.1145/3510003.3510067

[71] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. AutoTransform: Automated code transformation to support modern code review process. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*. ACM, 237–248. https://doi.org/10.1145/3510003.3510067

[72] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural machine translation. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE '19)*. IEEE, 25–36.

[73] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology* 28, 4 (2019), 1–29.

[74] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, 2291–2302. https://doi.org/10.1145/3510003.3510621

[75] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017), 1–11.

[76] Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. 2022. What do they capture? A structural analysis of pre-trained language models for source code. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. 2377–2388. https://doi.org/10.1145/3510003.3510050

[77] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2022. No more fine-tuning? An experimental evaluation of prompt tuning in code intelligence. *arXiv preprint arXiv:2207.11680* (2022).

[78] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging pre-trained models and downstream tasks for source code understanding. In *Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. IEEE, 287–298.

[79] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).

[80] Yu Wang, Ke Wang, and Linzhang Wang. 2021. WheaCha: A method for explaining the predictions of code summarization models. *arXiv preprint arXiv:2102.04625* (2021).

[81] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).

[82] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2019. HuggingFace's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).

[83] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural attack for pre-trained models of code. In *Proceedings of the 44th International Conference on Software Engineering*. 1482–1493.

[84] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 39–51.

[85] Beiqi Zhang, Peng Liang, Xiyu Zhou, Aakash Ahmad, and Muhammad Waseem. 2023. Practices and challenges of using GitHub Copilot: An empirical study. *arXiv preprint arXiv:2303.08733* (2023).

[86] Fengyi Zhang, Bihuan Chen, Yufei Zhao, and Xin Peng. 2023. Slice-based code change representation learning. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER '23)*. IEEE, 319–330.

[87] Jiyang Zhang, Sheena Panthaplackel, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2022. CoditT5: Pretraining for source code and natural language editing. *arXiv preprint arXiv:2208.05446* (2022).

[88] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Diet code is healthy: Simplifying programs for pre-trained models of code. *arXiv preprint arXiv:2206.14390* (2022).

[89] Yu Zhou, Xiaoqing Zhang, Juanjuan Shen, Tingting Han, Taolue Chen, and Harald Gall. 2022. Adversarial robustness of deep code comment generation. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022), 1–30.

Received 5 February 2023; revised 19 November 2023; accepted 3 January 2024